Pérez Paitán Brent

Laboratorio S09. Mexi Banco: mensajería con RabbitMQ

Introducción

En esta práctica se trabajó con la comunicación asíncrona entre servicios utilizando RabbitMQ en el sistema Mexi Banco. El objetivo fue solucionar el problema que ocurría cuando el servicio de notificaciones no estaba disponible y una transferencia ya había realizado sus operaciones principales.

Para ello, se utilizó un exchange para publicar eventos, una cola durable para conservar los mensajes pendientes y un consumidor encargado de procesarlos. También se revisaron problemas relacionados con la entrega duplicada de mensajes y el procesamiento de eventos inválidos.

Finalmente, se estudiaron los mecanismos de idempotencia, las colas de mensajes no procesables (DLQ) y el patrón Transactional Outbox, que permite conservar los eventos pendientes cuando el broker no está disponible.

1. El problema original

En la versión inicial del sistema, el servicio de transferencia realizaba una llamada HTTP al servicio de notificaciones después de efectuar el cargo y el abono correspondientes. Esto generaba una dependencia directa entre ambos servicios, ya que la transferencia esperaba una respuesta de notificación para continuar con su proceso.

Cuando el servicio de notificaciones no estaba disponible, la transferencia podía haber realizado los movimientos de dinero y, aun así, responder con un error. Además, el proceso podía quedar en estado `ABONADA`, sin llegar a `COMPLETADA`.

Este comportamiento demuestra que una falla en un servicio secundario puede afectar una operación que ya realizó sus acciones principales.

Evidencia 1. Estado inicial y problema de la transferencia.



```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: appl
ication/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 013600b670c6
Content-Type: application/json
Date: Sat, 24 Oct 2026 23:32:29 GMT

{"id":9,"clabe":"002180000000000002","titular":"Beto","saldo":500}brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

La prueba permite observar el estado de la transferencia y los saldos de las cuentas después de realizar la operación con el servicio de notificaciones apagado. Estos datos permiten identificar si el dinero ya se había movido aunque la operación no hubiera alcanzado el estado final.

2. Publicación de eventos mediante RabbitMQ

Para solucionar el problema anterior, se incorporó RabbitMQ como intermediario de mensajes entre los servicios. En lugar de esperar una respuesta HTTP de notificación, el servicio de transferencia publica un evento que representa el resultado de la operación.

La configuración utiliza los siguientes componentes:

- **Exchange `transferencias`**: recibe los eventos publicados por el servicio de transferencia. Es de tipo `topic` y permite distribuir mensajes de acuerdo con su routing key.
- **Routing keys**: `transferencia.realizada` para las transferencias completadas y `transferencia.revertida` para las transferencias compensadas.
- **Cola `notificacion.transferencias`**: almacena los mensajes dirigidos al servicio de notificaciones.
- **Consumidor**: obtiene los eventos de la cola, procesa la información y guarda las notificaciones correspondientes.

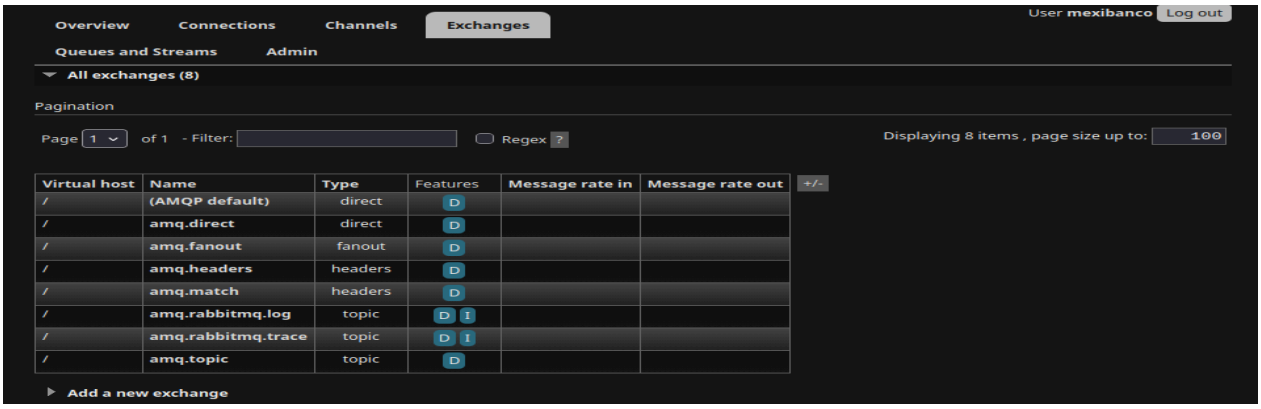
La cola es durable, por lo que su definición se conserva ante un reinicio del broker. Además, mientras el consumidor está apagado, los mensajes pueden permanecer pendientes en la cola para procesarse posteriormente.

Publicación y confirmación

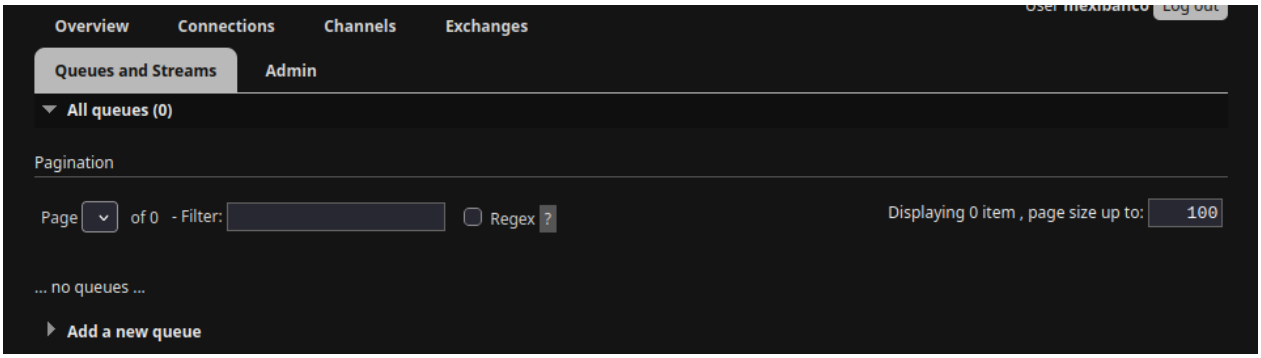
El publicador utiliza publisher confirms para conocer si RabbitMQ confirmó la recepción del mensaje. Esto permite detectar fallos de publicación y evita considerar enviado un evento cuando el broker no ha confirmado su recepción.

Sin embargo, esta confirmación no garantiza por sí sola que el servicio de notificaciones ya haya procesado el evento. La confirmación del broker y el procesamiento del consumidor son etapas distintas.

Evidencia 2. Configuración de RabbitMQ.



Evidencia 3. Publicación del evento.



3. Desacople en tiempo

Para comprobar el desacople entre los servicios, se apagó el contenedor de notificaciones y se realizaron solicitudes de transferencia. Durante este tiempo, el servicio de transferencia pudo publicar los eventos sin depender de que el consumidor estuviera ejecutándose.

Los mensajes publicados quedaron pendientes en la cola `notificacion.transferencias`. Cuando se volvió a iniciar el servicio de notificaciones, el consumidor pudo recuperar los mensajes y generar los avisos correspondientes.

Este comportamiento permite que ambos servicios trabajen en momentos diferentes, sin que una interrupción temporal del consumidor obligue a detener las transferencias.

Prueba con tres transferencias

Se realizaron tres transferencias mientras el servicio de notificaciones estaba apagado. De acuerdo con el procedimiento de la práctica, se verificó la respuesta de las solicitudes y la cantidad de mensajes pendientes en RabbitMQ.

Evidencia 4. Transferencias realizadas con notificaciones apagadas.

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":4,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50.00,"fecha":"2026-10-25T03:
13:23.657498Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actualizada
":"2026-10-25T03:13:24.235757Z"}]brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -s http://localhost:8080/api/cuentas/002180000000000001
{"id":8,"clabe":"002180000000000001","titular":"Ana","saldo":950.00}brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -s http://localhost:8080/api/cuentas/002180000000000002
{"id":9,"clabe":"002180000000000002","titular":"Beto","saldo":550.00}brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Typ
e: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 25 Oct 2026 03:54:33 GMT

{"id":5,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50,"fecha":"2026-10-25T03:54:3
3.656812843Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":900.00,"intentosDeCompensacion":0,"actualiza
da":"2026-10-25T03:54:34.063771603Z","eventoId":"697e47d1-96fa-43ac-81f1-3fe7d0c03357"}brent@pc-bda-bpp:~/SDistribuid
os/mexi-banco$
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

Después se volvió a iniciar el servicio de notificaciones. El consumidor procesó los mensajes pendientes y generó las notificaciones asociadas a las transferencias.

Evidencia 5. Notificaciones después de recuperar el servicio.

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ for m in 11 12 13; do                    # 2. tres transferencias,
de 11, 12 y 13
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"002180000000000001\",\"claveDestino\":\"002180000000000002\",\"monto\":$m}"
done
201
201
201
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

También se contempla el evento `TransferenciaRevertida`, que permite comunicar que una transferencia fue compensada. De esta manera, el sistema puede informar tanto de las operaciones completadas como de aquellas que fueron revertidas.

Evidencia 6. Evento de transferencia revertida.

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready
 messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     0       0
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

4. El duplicado: idempotencia del consumidor

En los sistemas de mensajería, un evento puede entregarse más de una vez. Esto puede ocurrir, por ejemplo, cuando el consumidor procesa un mensaje, pero falla antes de

confirmar su recepción. RabbitMQ puede volver a entregar el mensaje porque no recibió el reconocimiento correspondiente.

Si el consumidor genera notificaciones cada vez que recibe el evento, una entrega duplicada puede producir avisos repetidos para la misma transferencia.

Para resolver este problema, se incorporó un mecanismo de idempotencia basado en el identificador único `eventoId`. Este identificador se mantiene igual cuando se vuelve a publicar el mismo evento y permite reconocer si ya fue procesado.

Se agregó un registro de eventos procesados en la base de datos. Antes de generar las notificaciones, el consumidor intenta registrar el identificador del evento. Si el registro se realiza por primera vez, continúa con el procesamiento; si el identificador ya existe, evita generar nuevamente las notificaciones.

El registro del evento y el guardado de las notificaciones deben participar en una misma transacción de base de datos. Así, si el guardado falla, también se revierte el registro del evento y puede intentarse nuevamente su procesamiento.

Evidencia 7. Comportamiento antes de la idempotencia.

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Typ
e: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":150}'
HTTP/1.1 502 Bad Gateway
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sun, 25 Oct 2026 04:16:42 GMT

{"type":"about:blank","title":"Transferencia no realizada","status":502,"detail":"No se pudo abonar a la CLABE 002180
000000000002 y la devolucion de 150 a la CLABE 002180000000000001 tampoco pudo hacerse todavia. El dinero salio de or
igen; el banco reintentara la devolucion automaticamente.","instance":"/transferencias","transferencia":9,"estado":"C
OMPENSACION_PENDIENTE","motivo":"El abono a la CLABE 002180000000000002 fallo: Falla provocada con FALLAR_ABONO_A_CLA
BE=002180000000000002"}brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -s 'http://localhost:8080/api/notificaciones?clabe=0021800000000000
01'
[{"id":15,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00","fecha
":"2026-10-25T04:10:11.984761Z"},{"id":13,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 12,
 saldo resultante 877.00","fecha":"2026-10-25T04:10:10.961714Z"},{"id":11,"clabeCuenta":"002180000000000001","mensaje
":"TRANSFERENCIA_ENVIADA por 12, saldo resultante 877.00","fecha":"2026-10-25T04:10:10.894581Z"},{"id":9,"clabeCuenta
":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resultante 889.00","fecha":"2026-10-25T04:10:10
.655698Z"}]brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

Evidencia 8. Comportamiento después de la idempotencia.

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ docker compose logs --since=5m notificacion \
  | grep -E 'consumidor|ya procesado|aviso'
notificacion-1  | 2026-10-26T02:00:40.333Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransfere
ncias       : [consumidor] TransferenciaRealizada de la transferencia 11 (evento e6e590dd-df5d-45b3-a1ef-d3a2bca2ed56
, redelivered=true): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-26T02:00:40.344Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.Notificacion
Service     : [notificacion] Evento e6e590dd-df5d-45b3-a1ef-d3a2bca2ed56 ya procesado; 0 avisos nuevos
notificacion-1  | 2026-10-26T02:00:40.345Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransfere
ncias       : [consumidor] TransferenciaRealizada de la transferencia 11 (evento e6e590dd-df5d-45b3-a1ef-d3a2bca2ed56
, redelivered=false): 0 aviso(s) nuevo(s)
notificacion-1  | 2026-10-26T02:00:40.411Z  INFO 1 --- [notificacion] [ntContainer#0-2] m.m.notificacion.Notificacion
Service     : [notificacion] Evento e6e590dd-df5d-45b3-a1ef-d3a2bca2ed56 ya procesado; 0 avisos nuevos
notificacion-1  | 2026-10-26T02:00:40.412Z  INFO 1 --- [notificacion] [ntContainer#0-2] m.m.n.m.ConsumidorDeTransfere
ncias       : [consumidor] TransferenciaRealizada de la transferencia 11 (evento e6e590dd-df5d-45b3-a1ef-d3a2bca2ed56
, redelivered=true): 0 aviso(s) nuevo(s)
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

Con esta modificación, las entregas repetidas del mismo evento no deben producir un nuevo conjunto de notificaciones. Esto no impide que RabbitMQ vuelva a entregar un mensaje, sino que evita que esa entrega repetida tenga el mismo efecto en la aplicación.

5. La DLQ: mensaje envenenado

Un mensaje envenenado es aquel que no puede procesarse correctamente, por ejemplo, porque contiene un JSON mal formado o porque los datos del evento no cumplen con los requisitos del consumidor.

Si este mensaje se reencola continuamente, puede generar reintentos innecesarios y dificultar el procesamiento de los demás eventos. Para evitarlo, se configuró una Dead Letter Queue (DLQ), donde se pueden conservar los mensajes que fueron rechazados y revisar posteriormente su contenido.

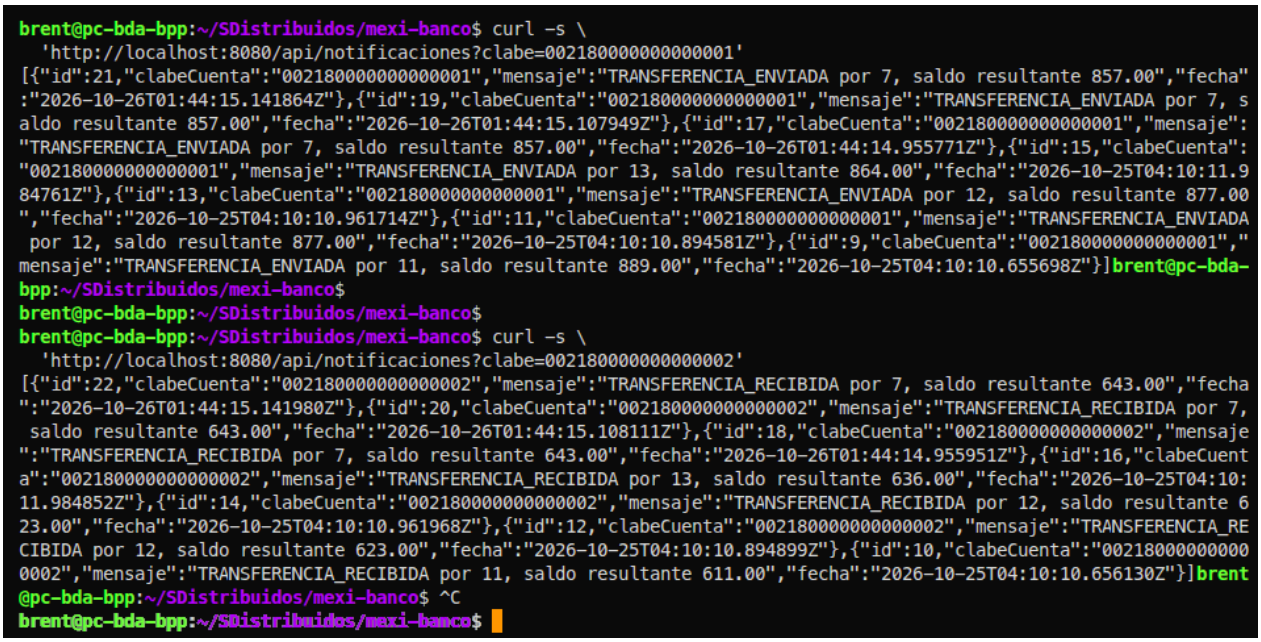
Prueba antes de configurar la DLQ

Se publicó desde el panel de RabbitMQ un mensaje inválido utilizando el exchange `transferencias`, la routing key `transferencia.realizada` y el siguiente contenido:

{"hola":

El contenido no corresponde a un JSON completo, por lo que el consumidor no puede convertirlo en un evento válido. Se observaron los logs del servicio y el estado de la cola para identificar cómo reaccionaba la configuración inicial ante este mensaje.

Evidencia 9. Mensaje envenenado antes de la corrección.



Configuración de la DLQ

Para separar los mensajes inválidos de los eventos normales, se configuraron tres componentes:

Componente	Nombre
Cola principal	`notificacion.transferencias`
Dead Letter Exchange	`transferencias.dlx`
Dead Letter Queue	`notificacion.transferencias.dlq`

La cola principal utiliza los argumentos `x-dead-letter-exchange` y `x-dead-letter-routing-key` para indicar dónde deben enviarse los mensajes rechazados. El exchange de dead lettering es de tipo `direct` y utiliza la clave `notificacion.transferencias` para dirigirlos a la DLQ.

En el consumidor se implementó el rechazo del mensaje inválido sin reencolarlo mediante `basicNack` con el parámetro `requeue` establecido en `false`. De esta forma, RabbitMQ puede trasladar el mensaje a la DLQ.

Los fallos de procesamiento deben distinguirse según su causa. Un mensaje mal formado no se soluciona repitiendo su procesamiento, mientras que un fallo temporal de la base de datos puede justificar un nuevo intento.

Evidencia 10. Mensaje rechazado en la DLQ.

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ docker compose exec rabbitmq rabbitmqctl list_queues \
  name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias.dlq 1       0
notificacion.transferencias     0       0
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

Verificación final

Después de configurar la DLQ, se publicó nuevamente un mensaje inválido y se verificó que apareciera en la cola de mensajes rechazados. También se realizó una transferencia normal para comprobar que el consumidor siguiera procesando eventos válidos.

Evidencia 11. Procesamiento normal después de la configuración.

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{
    "claveOrigen":"00218000000000001",
    "claveDestino":"00218000000000002",
    "monto":9
  }'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 26 Oct 2026 03:57:55 GMT

{"id":12,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":9,"fecha":"2026-10-26T03:57:5
5.703795027Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":840.00,"intentosDeCompensacion":0,"actualiza
da":"2026-10-26T03:57:55.805670691Z","eventoId":"87d42fdd-3350-49d5-b782-683384f15177"}brent@pc-bda-bpp:~/SDistribuid
os/mexi-banco$
```

```
brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$ curl -s \
  'http://localhost:8080/api/notificaciones?clabe=00218000000000002'
[{"id":26,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 9, saldo resultante 660.00","fecha
":"2026-10-26T03:57:55.997727Z"},{"id":24,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 8,
 saldo resultante 651.00","fecha":"2026-10-26T02:00:40.177073Z"},{"id":22,"clabeCuenta":"00218000000000002","mensaje
":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00","fecha":"2026-10-26T01:44:15.141980Z"},{"id":20,"clabeCuent
a":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00","fecha":"2026-10-26T01:44:1
5.108111Z"},{"id":18,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643
.00","fecha":"2026-10-26T01:44:14.955951Z"},{"id":16,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECI
BIDA por 13, saldo resultante 636.00","fecha":"2026-10-25T04:10:11.984852Z"},{"id":14,"clabeCuenta":"0021800000000000
02","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-10-25T04:10:10.961968Z"},{"id":1
2,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026
-10-25T04:10:10.894899Z"},{"id":10,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo
 resultante 611.00","fecha":"2026-10-25T04:10:10.656130Z"}]brent@pc-bda-bpp:~/SDistribuidos/mexi-banco$
```

La DLQ permite separar los mensajes que necesitan revisión sin mantenerlos bloqueando la cola principal. Sin embargo, los mensajes rechazados deben analizarse para identificar la causa del error y decidir si es necesario corregirlos o descartarlos.

6. Preguntas de cierre

6.1. Si RabbitMQ está caído cuando la saga llega a `COMPLETADA`, ¿qué pasa con el aviso en la versión implementada? ¿Por qué no lo arregla el publisher confirm?

En la versión sin Outbox, la transferencia puede llegar al estado `COMPLETADA` aunque RabbitMQ no esté disponible para recibir el evento. En ese caso, la publicación falla y el aviso puede perderse porque el evento no quedó guardado en una estructura persistente de pendientes dentro de la base de datos de transferencia. El publisher confirm permite saber si RabbitMQ confirmó un mensaje, pero no crea automáticamente un mecanismo para almacenarlo y publicarlo después. El patrón Outbox busca solucionar este problema al guardar el evento pendiente junto con el cambio de estado de la transferencia.

6.2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía se obtiene con cada orden?

El reconocimiento del mensaje debe realizarse después de guardar las notificaciones. Si el consumidor confirma el mensaje antes de guardar y posteriormente falla, RabbitMQ lo considera procesado y no lo vuelve a entregar, aunque los avisos no se hayan guardado. En cambio, si primero se guardan las notificaciones y después se confirma el mensaje, un fallo antes del reconocimiento puede provocar una nueva entrega. Por ello, es necesario combinar el reconocimiento posterior al guardado con un mecanismo de idempotencia que evite generar avisos duplicados.

6.3. Al quitar la llamada síncrona, ¿qué se ganó y qué se sacrificó?

Elección: utilizar comunicación asíncrona mediante RabbitMQ para las notificaciones.

Porque: la transferencia puede continuar sin esperar una respuesta inmediata del servicio de notificaciones, lo que reduce la dependencia entre ambos servicios.

Sacrifico: la confirmación inmediata de que el usuario ya recibió su aviso. La notificación puede llegar después de que la transferencia se haya completado, y es necesario gestionar los mensajes pendientes y los posibles fallos de procesamiento.

Conclusiones

En esta práctica se identificó la importancia de reducir las dependencias directas entre servicios en un sistema distribuido. La comunicación asíncrona con RabbitMQ permite que las transferencias no dependan de la disponibilidad inmediata del servicio de notificaciones y que los mensajes permanezcan pendientes mientras el consumidor está apagado.

También se revisó que una entrega repetida puede provocar efectos duplicados, por lo que es necesario implementar idempotencia en el consumidor. Por otra parte, la DLQ permite separar los mensajes que no pueden procesarse para evitar reintentos innecesarios y facilitar su revisión.

Referencias

RabbitMQ. (s. f.). *RabbitMQ documentation*. https://www.rabbitmq.com/docs

Spring. (s. f.). *Spring AMQP*. https://docs.spring.io/spring-amqp/reference/

Ruiz, O. (2026). *Lab S09: Mexi Banco — Mensajería con RabbitMQ* [Material de prácticas de Sistemas Distribuidos]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.